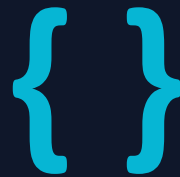


JavaScript Interactivity & DOM Manipulation

Making Web Pages Come Alive with JavaScript

01 Understanding the DOM

- 02 Selecting & Manipulating Elements
- 03 Event Handling
- 04 Dynamic Styling & Content
- 05 Basic Animations



```
document  
.querySelector()  
.addEventListener()
```



Ice Breaker!

Let's get to know each other before we dive in!



Who are you?

Share your name and one word that describes your relationship with coding so far.



Favorite Website?

Name one website or app you use daily — we'll try to understand how it uses the DOM!



Your Goal?

What's one thing you hope to build or achieve after learning JavaScript interactivity?



Tip: Think of any interactive feature on the web — buttons, forms, menus — that's the DOM at work!

Introduction & Learning Objectives

By the end of this session, students will be able to:

01

Understand the DOM

Describe the Document Object Model and how browsers represent HTML as a tree structure.

02

Select & Manipulate Elements

Use `querySelector`, `getElementById`, and other methods to find and modify page elements.

03

Handle Events

Attach click, submit, and keypress listeners to create truly interactive web experiences.

04

Update Style & Content

Dynamically change CSS properties, classes, and HTML content using JavaScript.

05

Create Basic Animations

Build simple animations using `setInterval`, `setTimeout`, and CSS transitions via JS.

Understanding the DOM

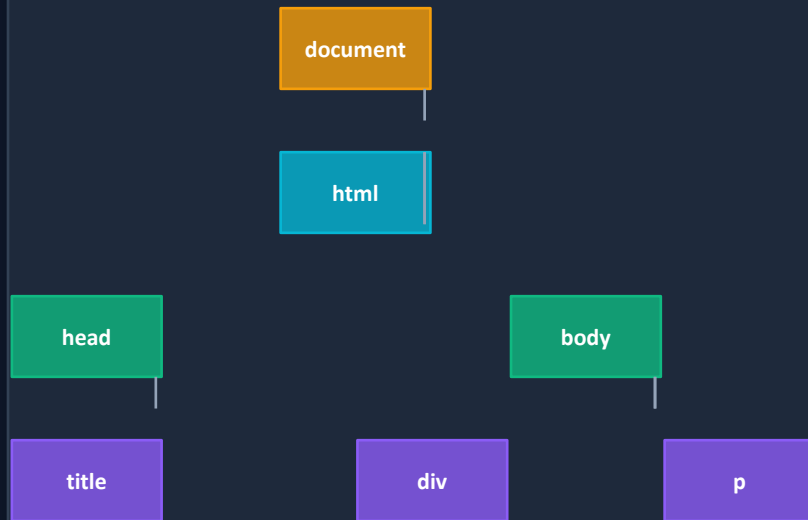
What is the DOM?

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree of objects. Every HTML element becomes a node that JavaScript can read, create, update, or delete.

How Browsers Use It

- Browser parses HTML → builds DOM tree
- JavaScript accesses DOM via `window.document`
- Changes to DOM update the visible page instantly
- DOM is language-independent (not just JS!)

DOM Tree Structure



02 Selecting & Manipulating Elements

Selection Methods

Method	Returns	Speed	Best For
<code>getElementById('id')</code>	Single Element	 Fastest	Unique elements
<code>querySelector('css')</code>	First Match	 Fast	Any CSS selector
<code>querySelectorAll('css')</code>	NodeList	 Moderate	Multiple elements
<code>getElementsByClassName()</code>	HTMLCollection	 Fast	Class-based
<code>getElementsByTagName()</code>	HTMLCollection	 Fast	Tag-based

Common Manipulations

Change Text

```
el.textContent = "Hello!"
```

Change HTML

```
el.innerHTML =  
"<b>Bold</b>"
```

Change Style

```
el.style.color = "red"
```

Add Class

```
el.classList.add("active")
```

Set Attribute

```
el.setAttribute("href", "url")
```

Event Handling (click, submit, keypress)

addEventListener Syntax

```
element.addEventListener(  
  "eventType",  
  function(event) {  
    // your code here  
  }  
);
```

Common Event Types

The Event Object

event.target — the element that fired the event
event.preventDefault() — stop default behavior
event.key — which key was pressed

"click"

Mouse button pressed & released on element

```
btn.addEventListener("click", fn)
```

"mouseover"

Mouse pointer enters element

```
el.addEventListener("mouseover", fn)
```

"submit"

Form data submitted by user

```
form.addEventListener("submit", fn)
```

"change"

Input value changed

```
select.addEventListener("change", fn)
```

"keypress"

Keyboard key pressed (deprecated → use keydown)

```
input.addEventListener("keydown", fn)
```

"load"

Page or resource finished loading

```
window.addEventListener("load", fn)
```

Dynamic Styling & Content Updates

Inline Styles

```
el.style.backgroundColor = "blue";  
el.style.fontSize = "20px";  
el.style.display = "none";
```

✓ Direct, immediate effect

⚠ Overrides CSS, hard to maintain

CSS Classes (Recommended)

```
el.classList.add("active");  
el.classList.remove("hidden");  
el.classList.toggle("open");
```

✓ Clean separation, reusable

⚠ Requires predefined CSS

CSS Custom Properties

```
el.style.setProperty(  
  "--main-color", "#ff0"
```

✓ Theme-wide changes possible

⚠ Slightly more complex syntax

Content Update Methods

textContent

Plain text only — safe from XSS

Low

innerHTML

HTML markup — dynamic content

⚠ XSS risk

innerText

Visible text (respects CSS)

Low

insertAdjacentHTML()

Insert HTML at specific position

⚠ XSS risk

Basic Animations with JavaScript

setTimeout & setInterval

```
let pos = 0;
const timer = setInterval(() => {
  el.style.left = pos + "px";
  pos += 5;
  if (pos > 300) clearInterval(timer);
}, 16);
```

Classic timer-based animations. Use `clearInterval()` to stop.

CSS Transitions via JS

```
el.style.transition = "all 0.5s ease";
el.style.transform = "translateX(200px)";
el.style.opacity = "0";
```

Let CSS handle the animation math — JS just triggers the change.

requestAnimationFrame

```
function animate() {
  pos += 2;
  el.style.left = pos + "px";
  if (pos < 300)
    requestAnimationFrame(animate);
}
requestAnimationFrame(animate);
```

Modern, browser-optimized. Syncs with display refresh rate (60fps).



Best Practice: Use `requestAnimationFrame` for smooth 60fps animations — it pauses automatically when tab is hidden, saving CPU.

Real-World Examples & Importance

Every interactive experience on the web uses DOM manipulation



Add to Cart

Amazon / Shopee

Clicking 'Add to Cart' updates cart count in the header without reloading — pure DOM manipulation.



Live Editing

Google Docs

Every keystroke updates the DOM in real-time, syncing your document across devices instantly.



Search Autocomplete

YouTube

Keypress events trigger API calls, and results appear below by dynamically inserting DOM nodes.



Game Logic

Browser Games

setInterval and requestAnimationFrame power character movement, physics, and score updates.



Drag & Drop

Trello / Notion

mousedown, mousemove, mouseup events track dragging; DOM updates move cards in real time.



Dark Mode Toggle

Any Modern Site

A single click event adds/removes a CSS class on the body, transforming the entire page theme.

Advantages & Disadvantages

Understanding trade-offs helps you write better, more responsible code

✓ ADVANTAGES

Instant User Feedback

Pages respond without full reloads — faster, more fluid UX.

No Framework Required

Pure JS DOM skills work in any browser, zero dependencies.

Full Control

Access and modify any element, attribute, or style precisely.

Foundation for Frameworks

React, Vue, Angular all sit on top of DOM manipulation concepts.

Richer Interactivity

Games, animations, real-time apps are only possible via DOM.

⚠ DISADVANTAGES

Security Risks (XSS)

Using innerHTML with user input can inject malicious scripts.

Performance Bottlenecks

Excessive reflows/repaints from DOM changes slow the page.

Harder to Debug

Async events and dynamic changes can create tricky race conditions.

Accessibility Concerns

Dynamic content may confuse screen readers if ARIA roles are missing.

Browser Inconsistencies

Older browsers may handle some DOM APIs differently.



Key Takeaways

What you should walk away knowing

1

The DOM is your gateway to the page

Every HTML element is an object you can select, read, and modify. master the tree structure concept first.

2

Prefer `querySelector` for flexibility

While `getElementById` is fastest, `querySelector` with CSS selectors gives you power and readability for complex selections.

3

`addEventListener` is the gold standard

Never use inline `onclick`. `addEventListener` keeps your HTML clean and lets you attach multiple listeners to one element.

4

`classList` over inline styles

Toggle CSS classes via `classList.add/remove/toggle` — this keeps your styles in CSS where they belong.

5

Use `requestAnimationFrame` for animations

It's browser-optimized, pauses when tab is not visible, and delivers smooth 60fps — always prefer it over `setInterval`.

6

Security & Performance matter

Sanitize innerHTML inputs, batch your DOM updates, and consider accessibility — responsive pages must be inclusive pages.

Conclusion

JavaScript DOM Manipulation is the core skill that transforms static HTML pages into living, breathing applications. Every modern web experience — from a simple button click to complex drag-and-drop interfaces — relies on these fundamental concepts you've learned today.



Practice Daily

Build small projects: a to-do list, a color picker, a quiz app. Each one will reinforce today's concepts.



Experiment Freely

Open DevTools on any website and run DOM commands in the console. Real sites are your playground.



Level Up

Once comfortable, explore Fetch API + DOM (dynamic data), then React/Vue which build on these foundations.

"The best way to learn programming is to write programs." — Dennis Ritchie



Suggested Videos to Watch

Watch these to deepen your understanding of JavaScript & DOM — all available on YouTube

JavaScript DOM Crash Course

Beginner

Traversy Media

Part 1-4 series — covers all DOM fundamentals from selection to events. Beginner-friendly.

JavaScript DOM Tutorial Series

Beginner

The Net Ninja

12-part playlist walking through every DOM method with live coding examples.

JavaScript DOM in 100 Seconds

Quick

Fireship

Fast-paced visual overview of the DOM concept — great for conceptual understanding first.

CSS + JavaScript Animations

Intermediate

Kevin Powell

Covers CSS transitions triggered by JS class toggling — perfect for Slide 5 content.

JavaScript Events Explained

Intermediate

Web Dev Simplified

Deep dive into event listeners, bubbling, capturing, and the event object.

Build Interactive Projects

Project

Clement Mihailescu

Builds real projects (Typing Speed Test, Calculator) using pure DOM manipulation.



Search these exact titles on YouTube for free access



References

Official Documentation

- MDN Web Docs — Introduction to the DOM | developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction
- MDN — `Document.querySelector()` | developer.mozilla.org/en-US/docs/Web/API/Document/querySelector
- MDN — `EventTarget.addEventListener()` | developer.mozilla.org/en-US/docs/Web/API/EventTarget/addEventListener
- MDN — `Window.requestAnimationFrame()` | developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame

Books & Learning Resources

- Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media.
- Haverbeke, M. (2018). Eloquent JavaScript (3rd ed.) — eloquentjavascript.net (Free Online)
- freeCodeCamp — JavaScript Algorithms and Data Structures | freecodecamp.org

Additional Online Resources

- javascript.info — The Modern JavaScript Tutorial — Comprehensive DOM section with interactive examples.
- W3Schools DOM Tutorial | w3schools.com/js/js_htmlDOM.asp — Quick reference with try-it editors.